AirControl AI

Smart Hands-Free Computer Control System

Controlling your computer with hand gestures, using a laptop camera and Computer Vision

Project Proposal

1. Executive Summary

AirControl AI is a desktop application that lets users control selected computer functions using hand and finger gestures, without touching the mouse or keyboard. The system uses the laptop's built-in camera and Computer Vision to detect the user's hand, recognize gestures, and translate each gesture into a specific computer command.

The first version focuses on media control: adjusting system volume, and playing, pausing, forwarding and rewinding videos. The system is designed to be extensible, so more modes such as presentation control can be added later.

2. Project Idea

Imagine watching a video on YouTube while taking notes. Instead of dropping your pen and reaching for the mouse to lower the volume, you simply make a gesture in front of the camera. The software sees the gesture, understands it, and performs the action.

The goal is to make interacting with a computer easier, faster, and more natural in situations where the hands are busy.

3. Problem Statement

Traditional computer interaction depends on the mouse and keyboard, which is impractical in many everyday situations:

- A student watches a lecture while taking handwritten notes.
- A person cooks while following a recipe video on a laptop.
- A presenter wants to change slides without walking back to the computer.
- A user wants to adjust volume or playback quickly without interrupting their current task.

AirControl AI addresses this by offering an alternative, touch-free way to control common computer functions.

4. Project Objectives

- Develop software that recognizes hand movement using a laptop camera.
- Use Computer Vision to detect the hand and locate fingers and joints.
- Recognize a defined set of hand gestures.
- Map each gesture to a specific computer command.
- Support volume control and video play, pause, forward and rewind.
- Provide a simple interface to start, stop and customize the system.
- Reduce the need for mouse and keyboard for the supported tasks.

5. How It Works

The system works as a five-stage pipeline that runs continuously while control mode is enabled:

Stage	Name	Description
1	Camera Input	The laptop camera continuously captures frames of the user's hand.
2	Hand Detection	A hand-tracking library detects the hand and locates fingers and joints (landmarks).
3	Gesture Recognition	Finger positions and hand shape are analyzed to identify the gesture, e.g. open palm vs. closed fist.
4	Command Mapping	The recognized gesture is matched to its assigned command.
5	Action Execution	The command is executed on the computer, and the system keeps listening for new gestures.

6. Main Features

Features are grouped so the project can be built incrementally.

6.1 Media Control (first priority)

Proposed Gesture	Command
Thumb up	Increase volume
Thumb down	Decrease volume
Closed fist	Pause video
Open palm	Play video
Hand moves right	Forward
Hand moves left	Rewind

Note: These gestures are proposals and will be tested and adjusted during implementation to find the most accurate and comfortable set. Video control depends on how commands are sent to the browser or media player, so not every function will work automatically with every application without extra configuration.

6.2 System Volume Control

The application controls the device's system volume, not only a single video's volume. It can therefore be used with YouTube, audio files, and other applications, provided the control method supports them.

6.3 Presentation Control

- Gesture for the next slide.
- Gesture for the previous slide.
- Gesture to start or end the slideshow.

7. Role of AI and Computer Vision

Computer Vision enables the computer to analyze images and video and extract information from them. Here it is used to detect the hand and track finger positions in real time.

Hand Gesture Recognition classifies the detected hand shape and motion into a specific gesture, which is then mapped to a command.

The project starts with rule-based logic built on finger landmark positions. If needed, it can later evolve to use a trained gesture-classification model. Training an AI model from scratch is not required at the start; ready-made hand-tracking tools are used as the foundation.

8. Technologies Used

Technology	Purpose
Python	Main programming language
OpenCV	Camera access and image processing
MediaPipe	Hand detection and finger landmark tracking
Pycaw	Windows system volume control
PyAutoGUI	Sending keyboard/control commands to the computer
PySide6 or Tkinter	Desktop user interface

Why Python? It is well suited to projects that combine Computer Vision, automation and AI, and offers mature libraries for each part.

Are HTML, CSS and JavaScript needed? Not necessarily. Since the main goal is controlling the laptop itself, a Python desktop application is recommended. A web interface remains possible later, paired with a backend component for computer control.

9. User Interface

The application window will include:

- **Start Camera / Stop Camera:** turn the camera on or off.
- **Enable Control / Disable Control:** start or stop accepting gesture commands.
- **Current Gesture:** shows the gesture currently recognized.
- **Volume Level:** shows the current system volume.
- **Control Mode:** select the active mode.
- **Gesture Settings:** customize gestures and commands.
- **Live camera view** with landmarks drawn on the fingers to visualize tracking.

10. Safety and Accuracy

The system must not execute commands simply because the user moved their hand naturally (for example, while writing). The following mechanisms address this:

Mechanism	Description
Activation Gesture	A dedicated gesture that turns control mode on.
Control Toggle	A button to enable or disable gesture control.
Gesture Stability	The gesture must be held steadily for a short time before the command runs.
Cooldown	Prevents a command from repeating too quickly by accident.
Lighting Handling	Testing under different lighting conditions.
Privacy	Camera frames are processed locally; no video is uploaded to external servers.

These will be added gradually once the core functions are working.

11. Development Plan

Phase	Task
1	Set up the Python environment and install libraries
2	Run the camera and detect the hand
3	Track fingers and recognize gestures
4	Connect gestures to volume control
5	Add video play and pause control
6	Add forward and rewind
7	Design the application interface
8	Test functions and handle errors
9	Prepare a runnable version and present the project

Development starts with simple functions and moves to more complex ones only after the earlier ones work.

12. Minimum Viable Product (MVP)

The first version will be considered complete when it can:

- Detect the hand through the laptop camera.
- Recognize a small set of defined gestures reliably.
- Control system volume.
- Play, pause, forward and rewind video in supported environments.
- Show system status and the recognized gesture.
- Let the user enable and disable control.

A feature is only counted as done after it has been implemented and actually tested.

13. Future Development

- **Custom Gestures:** users choose their own gestures and assign commands.
- **Smart Modes:** Media Mode, Study Mode, Presentation Mode, Computer Mode.
- **Advanced Gesture Recognition:** better handling of similar-looking gestures.
- **Application Integration:** support for more apps, depending on each app's control options.
- **Personalized Settings:** save chosen gestures and sensitivity levels.

14. Expected Outcome

A working desktop application that detects the hand through the laptop camera, recognizes defined gestures, controls system volume, sends playback commands in supported environments, displays its status, and lets the user enable or disable control.

15. What Makes AirControl AI Stand Out

Rather than a single fixed-function program, AirControl AI is an extensible system: it can start with volume control and grow to video, presentations, and more. It also gives hands-on experience across several areas:

- Working with external libraries
- Processing live camera data
- Designing gesture-recognition logic
- Executing commands on the operating system
- Building a user interface
- Testing and debugging